L1 Engineer's Guide #2 — Tempo · Mimir · Loki · Span-Metrics

Terminology, config files and the step-by-step process for Java / .NET / Python apps on VMs and EKS

Trace3 Delivery — Pacific Life AppDynamics Exit

L1 Engineer's Guide #2 — Tempo, Mimir, Loki & Span-Metrics: terminology, config files, and the step-by-step process for Java / .NET / Python on VMs and EKS

> Companion to Guide #1 (`08-l1-engineer-guide.md`). Guide #1 explained the *why* and the big picture. This one is the *how*: what each backend is, what its config looks like, which lines matter, and the exact order of steps to get one application's traces, metrics and logs flowing — for each runtime, on a VM or on EKS.

---

1. Terminology you will meet in the config files

1.1 Signals and protocol

Term	Meaning	Where you see it
Signal	One of the three telemetry types: **traces**, **metrics**, **logs**. Every Alloy component declares which signals it handles (`output { traces = [...] metrics = [...] logs = [...] }`).	Alloy configs
OTLP	OpenTelemetry Protocol. gRPC on **4317**, HTTP/protobuf on **4318**. Carries all three signals.	`OTEL_EXPORTER_OTLP_ENDPOINT`, `otelcol.receiver.otlp`
Resource	"Who is sending": a set of attributes attached to *everything* a process emits. Key ones: `service.name`, `service.namespace`, `deployment.environment`, `app.id`, `host.name`, `k8s.pod.name`.	`OTEL_RESOURCE_ATTRIBUTES`, `context = "resource"` in OTTL
Span	One timed operation. Fields: `name`, `kind` (SERVER / CLIENT / CONSUMER / PRODUCER / INTERNAL), `status` (UNSET / OK / ERROR), `attributes`, `trace_id`, `span_id`, `parent_span_id`.	Tempo, OTTL `context = "span"`
Root span / entry span	The first span of a trace inside a service — a SERVER span for HTTP, CONSUMER for a queue. **This is what AppD called a Business Transaction entry point.**	BT rules target `kind == SPAN_KIND_SERVER`
Semantic conventions (semconv)	The agreed attribute names. They changed in 2023: old `http.target` / `http.status_code` / `http.url` → new `url.path` / `http.response.status_code` / `url.full`. **Agents of different versions emit different names.** Our pipeline normalises old → new (`enrich` block).	`alloy/eks/config.alloy`
Instrumentation scope	Which library produced the span (e.g. `io.opentelemetry.tomcat-10.0`, `opentelemetry.instrumentation.flask`). Useful to see *which* auto-instrumentation fired.	Tempo span details
Exemplar	A trace_id attached to a metric sample so you can jump from a latency histogram bucket straight to an example trace.	Tempo metrics-generator `send_exemplars: true`, Grafana "exemplars" toggle

Term	Meaning	Where you see it
Baggage	Key/values propagated across services with the trace context. Rarely needed here.	—
W3C `traceparent`	The HTTP header that carries trace context between services. All three agents use it by default — that is how one trace spans Java → .NET → Python.	HTTP headers

1.2 Backends in Grafana Cloud

Term	Meaning	Config surface
Tempo	Trace store. Write path: OTLP in. Read path: **TraceQL**. Has the **metrics-generator**.	Grafana Cloud: portal toggles. Self-hosted: `tempo.yaml`
Metrics-generator	Tempo component that derives metrics from spans **as they arrive**: `span-metrics` processor (RED per service/span name) and `service-graphs` processor (edges between services). Writes to Mimir via remote-write.	`metrics_generator:` + `overrides:` ⋯ `processors: [span-metrics, service-graphs]`
Span-metrics	The metric series the generator writes: `traces_spanmetrics_calls_total`, `traces_spanmetrics_latency_bucket/_sum/_count`, `traces_spanmetrics_size_total`. Labels: `service`, `span_name`, `span_kind`, `status_code` + any configured `dimensions`. **= AppD BT metrics.**	Mimir
Service graph metrics	`traces_service_graph_request_total`, `⋯_failed_total`, `⋯_server_seconds_bucket` with `client` / `server` labels. Powers the service map (= AppD flow map).	Mimir
Mimir	Prometheus-compatible, horizontally scalable metrics store. Query language **PromQL**. Receives: span-metrics (from Tempo), host/runtime metrics (from Alloy), OTLP metrics (from agents).	Grafana Cloud: managed; you control *what you send* and limits/cardinality
Remote write	The Prometheus push protocol Alloy and Tempo use to write into Mimir.	`prometheus.remote_write`, Tempo `storage.remote_write`
Cardinality	Number of distinct label combinations per metric. `span_name` with raw IDs (`/api/quote/123`) explodes cardinality and cost. The pipeline's global guard rewrites `/123` → `/{id}`.	Mimir limits (`max_global_series_per_user`), Alloy BT rules
Loki	Log store. Indexes **labels** (few, low-cardinality) and keeps everything else as **structured metadata** or in the line. Query language **LogQL**. OTLP ingest endpoint `/otlp`.	Grafana Cloud: managed; OTLP resource-attribute → label mapping configurable
Label vs structured metadata (Loki)	Labels = indexed stream identity (`service_name`, `deployment_environment`, `app_id`). Structured metadata = per-line key/values not indexed (`trace_id`, `span_id`, `severity`). Putting `trace_id` in a *label* would be a cardinality disaster.	Loki `otlp_config`
TraceQL / PromQL / LogQL	Query languages for Tempo / Mimir / Loki. Cheatsheet in §8.	Grafana Explore
Application Observability	Grafana Cloud app that reads span-metrics + service-graph metrics and shows RED per service/operation and the service map. Needs `service.namespace` set (= our `app.id`).	Grafana Cloud UI
Data source UID	Stable ID Grafana uses to reference Tempo/Mimir/Loki in dashboards and alert rules. Terraform needs them.	`prometheus_datasource_uid`, etc.

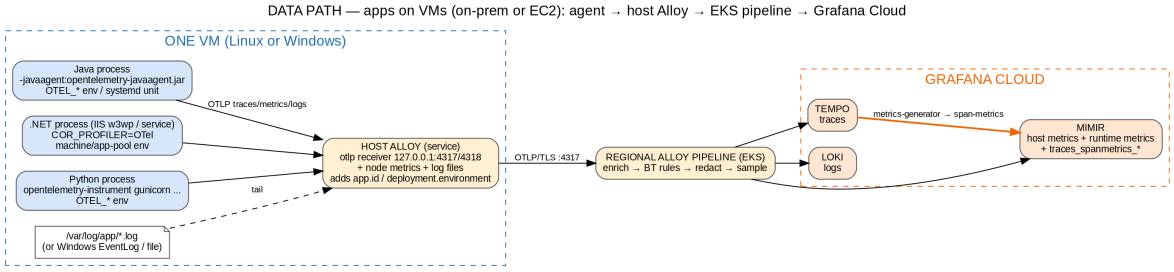
1.3 Alloy / collector terms

Term	Meaning
Component	One block in a `.alloy` file: `otelcol.receiver.otlp "ingress" { … }`. Type + label. Components reference each other by `type.label.export` (e.g. `otelcol.processor.batch.export.input`).
Receiver / Processor / Exporter / Connector	Receive telemetry in → transform it → send it out. A **connector** is both exporter and receiver (e.g. `otelcol.connector.spanmetrics` consumes traces and emits metrics).
OTTL statement	`function(args) where condition`. `set`, `delete_key`, `replace_pattern`, `replace_all_patterns`, `Concat`, `ExtractPatterns`, `IsMatch`. Runs in a `context` (`resource`, `span`, `log`, `metric`).
`error_mode = "ignore"`	A statement that errors (e.g. attribute missing) is skipped instead of dropping the batch. Always set it.
Tail sampling	Hold complete traces for `decision_wait`, then keep/drop by policy. Requires all spans of a trace to reach the *same* collector replica (load-balance by trace ID when >1 replica).
Head sampling	Decide at the *agent* when the trace starts (`OTEL_TRACES_SAMPLER`). We set `parentbased_always_on` so the pipeline decides.
Batch processor	Buffers and sends in chunks; always the last processor before exporters.
Memory limiter	First processor; refuses data when the pod is near its memory limit instead of crashing.
DaemonSet (EKS)	One Alloy pod per worker node — the Kubernetes equivalent of "host Alloy". Apps send to the node-local pod via `hostPort` or to a cluster `Service`.
OTel Operator / Instrumentation CR (EKS)	Kubernetes operator that injects the Java/.NET/Python agent into pods based on an annotation. Alternative to baking the agent into images.

---

2. Where the pieces run — VM vs EKS

2.1 Apps on VMs (on-prem or EC2)

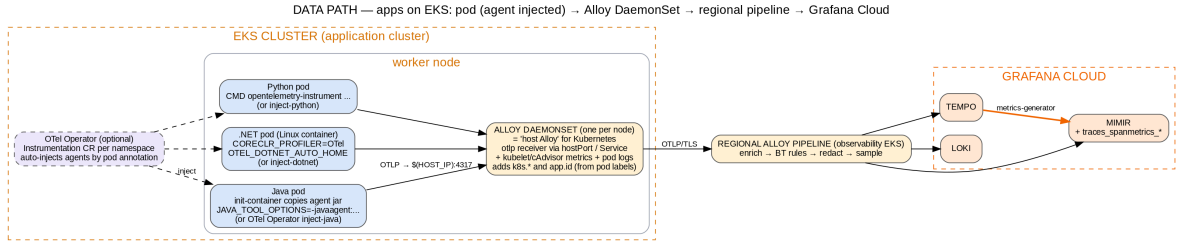


10 vm datapath

Layer	Linux VM	Windows VM
Agent config	`OTEL_*` env vars in systemd unit / `setenv.sh` / launch script	Machine-level or IIS app-pool environment variables (`[Environment]::SetEnvironmentVariable(..., 'Machine')`)
Local collector	Host Alloy as systemd service, listens `127.0.0.1:4317/4318`	Host Alloy as Windows service, same ports
Logs	Alloy tails files (`local.file_match` + `loki.source.file`) *or* agent sends OTLP logs	Alloy `loki.source.windowsevent` or file tail, *or* agent OTLP logs

Layer	Linux VM	Windows VM
Host metrics	`prometheus.exporter.unix`	`prometheus.exporter.windows`
Upstream	OTLP/TLS → regional pipeline NLB	same

2.2 Apps on EKS



11 eks datapath

Layer	How
Agent config	Env vars in the `Deployment` (`env:`), agent delivered by (a) baking into the image, (b) init-container copying the jar/profiler into a shared volume, or (c) **OTel Operator** injection via pod annotation
Local collector	**Alloy DaemonSet** (Helm chart `grafana/alloy`, `controller.type: daemonset`), OTLP receiver exposed via `hostPort` 4317 — apps use `OTEL_EXPORTER_OTLP_ENDPOINT=http://$(HOST_IP):4317` with `HOST_IP` from `status.hostIP`
Enrichment	`otelcol.processor.k8sattributes` adds `k8s.namespace.name`, `k8s.pod.name`, `k8s.deployment.name`; `app.id` comes from a pod label (`app.pacificlife.com/app-id`) mapped to a resource attribute
Logs	DaemonSet tails `/var/log/pods/**` (`loki.source.kubernetes` or file match) — agent OTLP logs also work
Metrics	kubelet / cAdvisor / kube-state-metrics via `prometheus.scrape`; runtime metrics (JVM, CLR, Python) via agent OTLP
Upstream	DaemonSet → regional pipeline (same OTLP/TLS)

---

3. Tempo — config and what matters

3.1 Grafana Cloud (what Pacific Life has)

You do not run Tempo; you configure it. The items that must be true before any app is migrated:

Setting	Where	Must be
Metrics-generator: **span-metrics** processor	Grafana Cloud portal → stack → Tempo → *Configure* → Metrics-generator	**Enabled** on prod *and* non-prod stacks (Discovery Q9)
Metrics-generator: **service-graphs** processor	same	Enabled (needed for service map / App O11y)
Span-metrics dimensions	same (or support ticket)	Defaults (`service`, `span_name`, `span_kind`, `status_code`) are enough for BT parity; optional extra: `bt.source`, `http.request.method`
Histogram buckets	default `[2ms … 15s]`	Default is fine; note p95 precision depends on buckets

Setting	Where	Must be
Ingest limits (bytes/s, traces/s)	portal → Tempo → limits	Sized for 10 apps (Discovery Q11)
Trace retention	portal	Default 30 days; confirm

How to verify (the only test that matters): in Explore → Mimir run `count(count by (service) (traces_spanmetrics_calls_total))` — if it returns a number, the generator is on. If "no data" with traces present in Tempo → generator off.

3.2 Self-hosted reference (`lab/tempo/tempo.yaml`) — annotated

```
server:
  http_listen_port: 3200                # Tempo API / Grafana datasource URL

distributor:
  receivers:                            # what Tempo accepts; we send OTLP from Alloy
    otlp:
      protocols:
        grpc: { endpoint: 0.0.0.0:4317 }
        http: { endpoint: 0.0.0.0:4318 }

ingester:
  max_block_duration: 5m                # how often in-memory traces are cut into blocks

compactor:
  compaction:
    block_retention: 24h                # trace retention (Grafana Cloud: portal setting)

storage:
  trace:
    backend: local                      # Grafana Cloud: object storage, managed
    wal:   { path: /var/tempo/wal }
    local: { path: /var/tempo/blocks }

metrics_generator:                      # <<< THE PART THAT MAKES BT METRICS
  registry:
    external_labels: { source: tempo }   # label added to every generated series
  storage:
    path: /var/tempo/generator/wal
    remote_write:                       # where the metrics go = Mimir / Prometheus
      - url: http://prometheus:9090/api/v1/write
        send_exemplars: true            # attach trace_ids to histogram samples
  processor:
    span_metrics:
      dimensions: [http.request.method, bt.source]   # extra labels from span attributes (keep few!)
      intrinsic_dimensions:
        service: true                   # -> label "service"
        span_name: true                 # -> label "span_name"   (= BT name after OTTL)
        span_kind: true
        status_code: true               # STATUS_CODE_OK / ERROR / UNSET
      # histogram_buckets: [0.002, 0.004, ⋯ 15]   # override only with a reason

overrides:
  defaults:
    metrics_generator:
      processors: [service-graphs, span-metrics, local-blocks]   # enable per tenant
```

What each processor gives you:

Processor	Produces	AppD equivalent
`span-metrics`	`traces_spanmetrics_*` per `service` × `span_name`	BT calls/min, errors/min, response time
`service-graphs`	`traces_service_graph_request_*` client→server edges	Flow map / tier dependencies

Processor	Produces	AppD equivalent
`local-blocks`	Lets TraceQL metrics queries (`{…} | rate()`) run over recent data	Ad-hoc BT analytics

---

4. Mimir — span-metrics, PromQL, limits

4.1 What arrives in Mimir for one app

Source	Metric family	Example series	Sent by
Tempo generator	`traces_spanmetrics_calls_total`	`{service="app01-quote-api", span_name="BT:quote", span_kind="SPAN_KIND_SERVER", status_code="STATUS_CODE_OK", source="tempo"}`	Tempo
Tempo generator	`traces_spanmetrics_latency_bucket` / `_sum` / `_count`	`{…, le="0.5"}`	Tempo
Tempo generator	`traces_service_graph_request_total`	`{client="app01-quote-api", server="rating-svc"}`	Tempo
Agent (OTLP metrics)	JVM: `jvm_memory_used_bytes`, `jvm_gc_duration_seconds`, `jvm_thread_count`; .NET: `process_runtime_dotnet_gc_*`, `http_server_request_duration_seconds`; Python: `http_server_duration`	`{service_name="app01-quote-api", app_id="APP-01", …}`	Alloy pipeline (`metrics` path)
Host Alloy	`node_cpu_seconds_total`, `node_memory_*`, `windows_*`	`{instance="vm-prd-01", app_id="APP-01"}`	Host Alloy remote-write
EKS DaemonSet	`container_cpu_usage_seconds_total`, `kube_pod_status_*`	`{namespace, pod}`	DaemonSet

Note the label-name difference: Tempo-generated series use `service`; OTLP-metrics series use `service_name` (dots become underscores). Dashboards must use the right one.

4.2 PromQL cookbook (copy these)

```
# Calls/min for every BT of one service (AppD "Calls per minute")
sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api"}[5m])) * 60

# Error % per BT (AppD "Errors per minute" as a ratio)
  sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api", status_code="STATUS_CODE_ERROR"}[5m]))
/ sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api"}[5m])) * 100

# p95 latency per BT in ms (AppD "95th percentile response time")
histogram_quantile(0.95, sum by (le, span_name) (rate(traces_spanmetrics_latency_bucket{service="app01-quote-api"}[5m]))) * 1000

# Average response time per BT (AppD "Average response time")
  sum by (span_name) (rate(traces_spanmetrics_latency_sum{service="app01-quote-api"}[5m]))
/ sum by (span_name) (rate(traces_spanmetrics_latency_count{service="app01-quote-api"}[5m])) * 1000

# All BTs of an APP_ID (service.namespace is copied from app.id by the pipeline)
sum by (service, span_name) (rate(traces_spanmetrics_calls_total{service_namespace="APP-01"}[5m])) * 60

# Downstream dependencies (AppD flow map edge)
sum by (client, server) (rate(traces_service_graph_request_total{client="app01-quote-api"}[5m]))

# Is the metrics-generator alive for this service? (should be > 0 when traffic exists)
count(traces_spanmetrics_calls_total{service="app01-quote-api"})

# Cardinality check: how many distinct span names does a service have? (>100 = fix BT rules)
count(count by (span_name) (traces_spanmetrics_calls_total{service="app01-quote-api"}))
```

4.3 Limits you can hit (Grafana Cloud)

Symptom	Limit	Fix
`429` / "per-user series limit" in pipeline or Tempo logs	`max_global_series_per_user`	Reduce `dimensions`; fix span-name cardinality; raise limit (cost)
Samples rejected "out of order"	multiple writers same series	One Tempo generator per stack; don't also run the Alloy spanmetrics connector *and* Tempo generator for the same traffic
Alert rule "no data"	query uses `service_name` instead of `service` (or vice-versa)	Check label names in Explore first

4.4 Optional: recording rules (pre-computed RED)

If dashboards for 343 apps get slow, pre-compute per-BT RED with Mimir recording rules (Terraform `grafana_rule_group` with `record` blocks, or Mimir ruler YAML):

```
groups:
  - name: appd-exit-red
    interval: 1m
    rules:
      - record: bt:calls_per_min:rate5m
        expr: sum by (service, span_name) (rate(traces_spanmetrics_calls_total[5m])) * 60
      - record: bt:error_ratio:rate5m
        expr: |
            sum by (service, span_name) (rate(traces_spanmetrics_calls_total{status_code="STATUS_CODE_ERROR"}[5m]))
          / sum by (service, span_name) (rate(traces_spanmetrics_calls_total[5m]))
      - record: bt:latency_p95_seconds:rate5m
        expr: histogram_quantile(0.95, sum by (le, service, span_name) (rate(traces_spanmetrics_latency_bucket[5m])))
```

---

5. Loki — labels, OTLP ingest, correlation

5.1 Two ways logs reach Loki

Path	When to use	trace_id correlation
Agent → OTLP logs (Java logback/log4j appender, .NET ILogger, Python logging)	App logs through a standard framework; you want structured logs with `trace_id` for free	Automatic — agent sets `trace_id` / `span_id` on each record
Alloy tails log files (`loki.source.file`, `loki.source.kubernetes`)	Legacy apps, third-party processes, when the OTLP log bridge is not enabled	Only if the app writes `trace_id` into the line (enable MDC injection: Java `OTEL_INSTRUMENTATION_COMMON_MDC_RESOURCE_ATTRIBUTES`, Python `OTEL_PYTHON_LOG_CORRELATION=true`) and Loki derived field parses it

5.2 Which resource attributes become Loki labels

Grafana Cloud Loki OTLP ingest promotes a default set to **labels**: `service.name`, `service.namespace`, `service.instance.id`, `deployment.environment`, `cloud.*`, `k8s.namespace.name`, `k8s.pod.name`, `k8s.container.name`, … (dots → underscores: `service_name`). Everything else (incl. `trace_id`, `span_id`, `severity_text`) becomes **structured metadata**. To add `app.id` as a label, self-hosted config is:

```
# loki.yaml (self-hosted reference; in Grafana Cloud this is a tenant setting / support request)
limits_config:
  allow_structured_metadata: true
  otlp_config:
    resource_attributes:
      attributes_config:
        - action: index_label
          attributes: [app.id, deployment.environment, service.name, service.namespace]
    log_attributes:
      - action: structured_metadata
        attributes: [trace_id, span_id, code.function]
```

Rule of thumb: **≤ 10 labels, none with unbounded values.** `trace_id` is never a label.

5.3 LogQL cookbook

```
# All logs for a service with a trace id present
{service_name="app01-quote-api"} |= "trace_id"

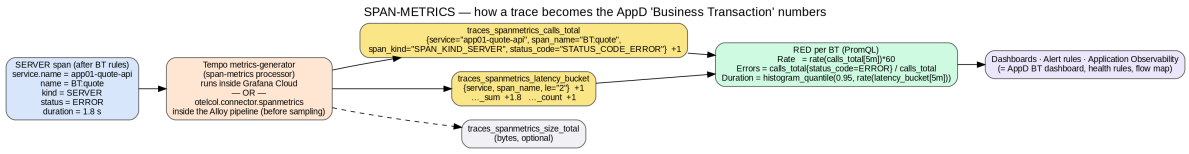
# Errors for one APP_ID (if app_id is a label) — otherwise filter on structured metadata
{app_id="APP-01"} | detected_level="error"
{service_name="app01-quote-api"} | severity_text="ERROR"

# Jump from a trace to its logs (Grafana does this via tracesToLogsV2; manual form):
{service_name="app01-quote-api"} | trace_id="74aefd2dcd81f81946bd5a03d1201e35"

# Redaction evidence for FPC §8
{service_name="app05-claims-portal"} |= "[SSN]"

# Log-based rate (rarely needed — span-metrics are the primary RED source)
sum by (service_name) (rate({app_id="APP-01"} | detected_level="error" [5m]))
```

6. Span-metrics — generator vs in-pipeline connector



12 spanmetrics

Two places can turn spans into metrics. Pick **one** per stream.

	Tempo metrics-generator (default)	`otelcol.connector.spanmetrics` **in Alloy pipeline**
Where	Inside Grafana Cloud, after ingest	Inside the EKS pipeline, **before** tail sampling
Counts	Only traces that survived sampling	**Every** span (exact AppD-style counts)
Config owner	Pacific Life (portal)	Trace3 PR to pipeline repo
Cost	Tempo ingest + Mimir series	Mimir series only (traces can be sampled harder)
Extra dimensions	Portal setting	Any attribute, per-rule
Recommendation	Start here (zero pipeline change)	Enable if Pacific Life needs exact counts or chooses aggressive sampling (Q11 decision)

In-pipeline connector (drop into `alloy/eks/config.alloy`; wire `enrich/bt_rules` output → connector input, connector metrics → batch):

```
otelcol.connector.spanmetrics "bt" {
  histogram {
    explicit {
      buckets = ["2ms", "5ms", "10ms", "25ms", "50ms", "100ms", "250ms", "500ms", "1s", "2s", "5s", "10s", "15s"]
    }
  }
  dimension { name = "http.request.method" }
  dimension { name = "bt.source" }
  dimension { name = "deployment.environment" }
  namespace                 = "traces.spanmetrics"     // yields traces_spanmetrics_* like Tempo
  metrics_flush_interval    = "15s"
  exemplars { enabled = true }
  output {
    metrics = [otelcol.processor.batch.export.input]
  }
}
// redact -> [otelcol.connector.spanmetrics.bt.input, otelcol.processor.tail_sampling.default.input]
```

Whichever you use, the series names and labels match, so dashboards/alerts in `terraform/modules/*` work unchanged.

---

7. Step-by-step: one application, end to end

Use with the Migration Pattern runbook (`03-⋯`) and the app's Functional Parity Checklist (FPC). Each step has a **verify** line — do not move on until it passes.

Step 0 — Platform prerequisites (once per stack, before any app)

#	Action	Verify
0.1	Confirm Tempo metrics-generator (span-metrics + service-graphs) on prod and non-prod	`count(traces_spanmetrics_calls_total)` returns data for *any* existing service
0.2	Collect data-source UIDs (Mimir, Tempo, Loki) per stack	Grafana → Connections → Data sources → copy UID
0.3	Regional pipeline reachable from app networks; Grafana Cloud OTLP credentials in K8s secret	`grpcurl -plaintext <nlb>:4317 list` from a VM; pipeline pod logs show no 401
0.4	Host Alloy (VM) or Alloy DaemonSet (EKS) present, OTLP receiver enabled	`curl -s localhost:12345/-/ready` on a VM; `kubectl -n observability get ds alloy` on EKS
0.5	ServiceNow webhook user + non-prod endpoint; Terraform service-account token	`terraform plan` succeeds in `envs/nonprod`

Step 1 — Identity: decide the names (per app)

Field	Rule	Example
`service.name`	One per deployable; lowercase, stable across versions	`app01-quote-api`
`service.namespace`	= `APP_ID` (pipeline copies from `app.id` if unset)	`APP-01`
`app.id`	CMDB APP_ID	`APP-01`
`deployment.environment`	`prod` / `nonprod` (match Grafana stack and alert matchers)	`nonprod`
`service.version`	Build/artifact version	`2026.06.3`

Write them into FPC header. Everything downstream keys on them.

Step 2 — Enable the agent (choose your runtime × hosting)

Common env (all runtimes, all hosting):

```
OTEL_SERVICE_NAME=app01-quote-api
OTEL_RESOURCE_ATTRIBUTES=app.id=APP-01,deployment.environment=nonprod,service.namespace=APP-01,service.version=2026.06.3
OTEL_EXPORTER_OTLP_PROTOCOL=grpc
OTEL_TRACES_SAMPLER=parentbased_always_on
OTEL_METRICS_EXPORTER=otlp
OTEL_LOGS_EXPORTER=otlp
# VM:  OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4317
# EKS: OTEL_EXPORTER_OTLP_ENDPOINT=http://$(HOST_IP):4317   (HOST_IP from fieldRef status.hostIP)
```

2A Java on VM (Linux, systemd / Tomcat)

```
# /etc/systemd/system/app01.service.d/otel.conf
[Service]
Environment="JAVA_TOOL_OPTIONS=-javaagent:/opt/otel/opentelemetry-javaagent.jar"
Environment="OTEL_SERVICE_NAME=app01-quote-api"
Environment="OTEL_RESOURCE_ATTRIBUTES=app.id=APP-01,deployment.environment=nonprod,service.namespace=APP-01"
Environment="OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4317"
Environment="OTEL_INSTRUMENTATION_LOGBACK_APPENDER_ENABLED=true"
Environment="OTEL_INSTRUMENTATION_HTTP_SERVER_CAPTURE_REQUEST_HEADERS=x-channel"
```

`sudo systemctl daemon-reload && sudo systemctl restart app01`. (Tomcat: put the same in `bin/setenv.sh` as `CATALINA_OPTS`.) Dual-run with AppD: keep the AppD `-javaagent` too.

2B Java on EKS (init-container, no image change)

```
spec:
  template:
    metadata:
      labels: { app.pacificlife.com/app-id: APP-01 }
    spec:
      initContainers:
        - name: otel-agent
          image: ghcr.io/open-telemetry/opentelemetry-java-instrumentation/opentelemetry-javaagent:2.9.0  # pin
          command: ["cp", "/javaagent.jar", "/otel/opentelemetry-javaagent.jar"]
          volumeMounts: [{ name: otel, mountPath: /otel }]
      containers:
        - name: app
          volumeMounts: [{ name: otel, mountPath: /otel }]
          env:
            - { name: JAVA_TOOL_OPTIONS, value: "-javaagent:/otel/opentelemetry-javaagent.jar" }
            - { name: HOST_IP, valueFrom: { fieldRef: { fieldPath: status.hostIP } } }
            - { name: OTEL_EXPORTER_OTLP_ENDPOINT, value: "http://$(HOST_IP):4317" }
            - { name: OTEL_SERVICE_NAME, value: app01-quote-api }
            - { name: OTEL_RESOURCE_ATTRIBUTES, value: "app.id=APP-01,deployment.environment=nonprod,service.namespace=APP-01" }
      volumes: [{ name: otel, emptyDir: {} }]
```

Alternative with the OTel Operator: annotate the pod `instrumentation.opentelemetry.io/inject-java: "true"` and define one `Instrumentation` CR per namespace with the exporter endpoint and resource attributes.

2C .NET on VM (Windows, IIS)

```
# as Administrator — AppD .NET agent must already be uninstalled (one profiler per process)
Import-Module .\OpenTelemetry.DotNet.Auto.psm1
Install-OpenTelemetryCore
Register-OpenTelemetryForIIS                       # sets COR_ENABLE_PROFILING / COR_PROFILER for w3wp
[Environment]::SetEnvironmentVariable('OTEL_SERVICE_NAME','app06-policy-web','Machine')
[Environment]::SetEnvironmentVariable('OTEL_RESOURCE_ATTRIBUTES','app.id=APP-06,deployment.environment=nonprod,service.namespace=AP
[Environment]::SetEnvironmentVariable('OTEL_EXPORTER_OTLP_ENDPOINT','http://127.0.0.1:4317','Machine')
[Environment]::SetEnvironmentVariable('OTEL_DOTNET_AUTO_TRACES_ASPNET_INSTRUMENTATION_CAPTURE_REQUEST_HEADERS','x-channel','Machine
iisreset
```

Windows service instead of IIS: `Register-OpenTelemetryForWindowsService -WindowsServiceName "PLPolicyWorker"`.

2D .NET on EKS (Linux container, .NET 6/8)

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0
ARG OTEL_VERSION=1.9.0
ADD https://github.com/open-telemetry/opentelemetry-dotnet-instrumentation/releases/download/v${OTEL_VERSION}/otel-dotnet-auto-inst
RUN apt-get update && apt-get install -y unzip curl && OTEL_DOTNET_AUTO_HOME=/otel sh /otel-install.sh
ENV OTEL_DOTNET_AUTO_HOME=/otel \
    CORECLR_ENABLE_PROFILING=1 \
    CORECLR_PROFILER={918728DD-259F-4A6A-AC2B-B85E1B658318} \
    CORECLR_PROFILER_PATH=/otel/linux-x64/OpenTelemetry.AutoInstrumentation.Native.so \
    DOTNET_ADDITIONAL_DEPS=/otel/AdditionalDeps \
    DOTNET_SHARED_STORE=/otel/store \
    DOTNET_STARTUP_HOOKS=/otel/net/OpenTelemetry.AutoInstrumentation.StartupHook.dll
COPY publish/ /app
WORKDIR /app
ENTRYPOINT ["dotnet", "PolicyWeb.dll"]
```

Set `OTEL_*` identity/env vars in the Deployment as in 2B. Operator alternative: `inject-dotnet: "true"`.

2E Python on VM (gunicorn, systemd)

```
[Service]
Environment="OTEL_SERVICE_NAME=app05-claims-portal"
Environment="OTEL_RESOURCE_ATTRIBUTES=app.id=APP-05,deployment.environment=nonprod,service.namespace=APP-05"
Environment="OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4317"
Environment="OTEL_PYTHON_LOG_CORRELATION=true"
Environment="OTEL_PYTHON_LOGGING_AUTO_INSTRUMENTATION_ENABLED=true"
Environment="OTEL_PYTHON_EXCLUDED_URLS=^/static/,^/health"
ExecStart=/opt/app05/.venv/bin/opentelemetry-instrument /opt/app05/.venv/bin/gunicorn app.wsgi:application -w 4 -b 0.0.0.0:8080
```

Install once: `pip install opentelemetry-distro opentelemetry-exporter-otlp && opentelemetry-bootstrap -a install`. gunicorn `--preload` → add `post_fork` hook (see `agents/python.md`).

2F Python on EKS

```
FROM python:3.12-slim
COPY requirements.txt .
RUN pip install -r requirements.txt opentelemetry-distro opentelemetry-exporter-otlp && opentelemetry-bootstrap -a install
COPY . /app
WORKDIR /app
CMD ["opentelemetry-instrument", "gunicorn", "app.wsgi:application", "-w", "4", "-b", "0.0.0.0:8080"]
```

Env vars in the Deployment as in 2B. Operator alternative: `inject-python: "true"`.

Verify Step 2 (all variants):

```
# agent started?   Java: log line "opentelemetry-javaagent - version: 2.x"   .NET: OTEL_DOTNET_AUTO_LOG_DIRECTORY logs   Python: no
# spans leaving?   host Alloy / DaemonSet log shows otlp receiver accepting; or run otelcol debug exporter in non-prod
# Tempo:           Explore → Tempo → { resource.service.name = "app01-quote-api" }   → at least one trace within 2 minutes
```

Step 3 — Verify the local collector (host Alloy / DaemonSet)

Check	VM	EKS
Alive	`curl -s localhost:12345/-/ready` → `Alloy is ready.`	`kubectl -n observability get pods -l app.kubernetes.io/name=alloy -o wide`
Receiver listening	`ss -ltnp | grep 4317`	`kubectl -n observability logs ds/alloy | grep 'Starting GRPC server'`
Forwarding OK	Alloy log has no `exporter … retry` / `queue is full`	same
Resource attributes added	Use `otelcol.exporter.debug` in non-prod or check a trace in Tempo shows `app.id`, `host.name` / `k8s.pod.name`	same

Step 4 — Verify the regional pipeline did its job

Open one trace in Tempo and check the SERVER span:

Expect	Means
`service.namespace = APP-01`	enrich worked
span name is `BT:…` (or a clean `http.route`)	BT rules fired; `bt.source` attribute says which rule
no `?query=` in `url.full`; no `[SSN]`-shaped raw data; no `authorization` header	redaction worked
error/slow traces always present, normal traces ~10 %	tail sampling policy as designed

If span names are unchanged: check attribute names (`url.path` vs `http.target`) — the normalisation block handles known cases; add any new ones there.

Step 5 — Verify span-metrics in Mimir

```
sum by (span_name, status_code) (rate(traces_spanmetrics_calls_total{service="app01-quote-api"}[5m])) * 60
```

Expect one row per BT. Compare with the AppD BT list from FPC §1: every in-scope AppD BT must have a corresponding `span_name`. Record calls/min side by side in FPC §2.

Step 6 — Verify logs in Loki and correlation

```
{service_name="app01-quote-api"} |= "trace_id"
```

Pick a line, copy the `trace_id`, open it in Tempo. Then from a Tempo span click *Logs for this span* — both directions must work (data source `tracesToLogsV2` and Loki derived field configured, see `lab/grafana/provisioning/datasources/datasources.yml`).

Step 7 — Dashboards, alerts, routing as code

1. Add the app to `terraform/envs/<env>/main.tf` (`business_transactions` from FPC §3, `assignment_group` from FPC §4).
2. PR → ADO pipeline → `plan` shows the objects → merge → `apply`.
3. Grafana → folder `APP-01 (<env>)`: dashboard shows the same BT rows as Step 5; rule group lists one rule per static health rule.
4. Enable `enable_routing_test = true`, wait for the incident, record number + group in FPC §4, set back to `false`.

Step 8 — Compare (non-prod) and record

Run for ≥ 3 business days (Java/Python dual-run; .NET side-by-side or canary). Export both sides; fill FPC §2 deltas. Pass: calls ±10 %, errors ±1 pt, p95 ±15 % or explained.

Step 9 — Prod cutover (CAB) and hypercare

Repeat Steps 2–6 in prod **without** the AppD agent. Verify within 15 minutes. Screenshot AppD Controller node list ("not reporting") → evidence. Start 10-business-day hypercare clock.

Step 10 — Evidence into the FPC

Trace screenshot, span-metrics table, routing incident number, redaction query result, Controller screenshot, rollback rehearsal date. Owner signs FPC §9.

---

8. Query cheat-sheet (one page)

Need	Where	Query
Any traces for a service	Tempo	`{ resource.service.name = "app01-quote-api" }`

Need	Where	Query
Error traces for a BT	Tempo	`{ resource.service.name = "app01-quote-api" && name = "BT:quote" && status = error }`
Slow traces	Tempo	`{ resource.service.name = "app01-quote-api" && duration > 2s }`
Traces for an APP_ID across services	Tempo	`{ resource.service.namespace = "APP-01" }`
Which rule renamed the span	Tempo	`{ span.bt.source = "header:x-channel" }`
Calls/min per BT	Mimir	`sum by (span_name)(rate(traces_spanmetrics_calls_total{service="X"}[5m]))*60`
Error % per BT	Mimir	`sum by (span_name)(rate(…{service="X",status_code="STATUS_CODE_ERROR"}[5m])) / sum by (span_name)(rate(…{service="X"}[5m]))*100`
p95 per BT	Mimir	`histogram_quantile(0.95, sum by (le,span_name) (rate(traces_spanmetrics_latency_bucket{service="X"}[5m])))`
Dependencies	Mimir	`sum by (client,server)(rate(traces_service_graph_request_total{client="X"}[5m]))`
JVM heap	Mimir	`jvm_memory_used_bytes{service_name="X", jvm_memory_type="heap"}`
Logs with trace ids	Loki	`{service_name="X"} |= "trace_id"`
Logs for one trace	Loki	`{service_name="X"} | trace_id="<id>"`
Redaction evidence	Loki	`{service_name="X"} |= "[SSN]"`

---

9. Troubleshooting matrix

Symptom	Most likely	Check / fix
No traces at all	Agent not loaded, or cannot reach 4317	Process start log; `OTEL_EXPORTER_OTLP_ENDPOINT`; VM: `ss -ltnp | grep 4317`; EKS: `HOST_IP` env present, DaemonSet `hostPort` set
Traces in non-prod but not prod	Firewall VM → NLB, or wrong stack credentials	Pipeline pod logs (401/403 = creds); host Alloy exporter retries = network
Traces present, **no** `traces_spanmetrics_*`	Metrics-generator disabled on that stack	Step 0.1 — portal; or enable in-pipeline connector (§6)
Span names are raw URLs (`/api/quote/123`)	BT rule didn't match: wrong `app.id`, old semconv attribute names, or wrong `kind`	Check `resource.app.id`, `url.path` present, `kind=SERVER`; add to normalisation block
Hundreds of span names for one service	Cardinality — IDs/UUIDs in names	Global guard regexes; add app-specific `replace_pattern`; Mimir series limit alerts
`status_code` always `UNSET` for 4xx	By OTel spec, server 4xx is **not** an error (unlike AppD)	Decide per app: alert on `http.response.status_code>=500` only, or add OTTL `set(status.code, STATUS_CODE_ERROR) where attributes["http.response.status_code"] >= 400` for parity
Logs arrive but no `trace_id`	Log bridge/MDC not enabled, or log written outside a span	Java: logback appender env; Python: `OTEL_PYTHON_LOG_CORRELATION=true`; .NET: ILogger bridge default on
Trace → Logs button empty	Data source link not configured, or time shift too narrow	`tracesToLogsV2` with `filterByTraceID`, `spanStartTimeShift: -5m`

Symptom	Most likely	Check / fix
Alert rule "No data"	Label mismatch (`service` vs `service_name`), or wrong datasource UID	Run the rule query in Explore on the same datasource
Alert fires, no ServiceNow incident	Policy matcher labels don't match rule labels; contact point auth	Alerting → Notification policies → *Preview routing*; Contact point → *Test*
.NET: nothing instrumented	AppD profiler still registered, or Framework < 4.6.2	`Get-ChildItem Env:COR_PROFILER*` ; uninstall AppD first; check runtime version
Python: some frameworks missing spans	Library not covered by `opentelemetry-bootstrap`	`pip list | grep opentelemetry-instrumentation-` ; unsupported framework → Code/Exception
EKS: all pods show `app.id=UNMAPPED`	Pod label → resource attribute mapping missing in DaemonSet	`otelcol.processor.k8sattributes` `extract { label { from="pod" key="app.pacificlife.com/app-id" tag_name="app.id" } }`

---

10. File index for this guide

File	Purpose
`alloy/host/config.alloy`	Host Alloy (VM): OTLP receiver, CMDB attrs, host metrics, logs → pipeline
`alloy/eks/config.alloy`	Regional pipeline: enrich + semconv normalisation, redact, tail sampling, export
`alloy/eks/bt-rules.alloy`	Per-app BT split rules (OTTL)
`alloy/eks/values.yaml`	Helm values — pipeline on EKS
`lab/tempo/tempo.yaml`	Self-hosted Tempo with metrics-generator (annotated in §3.2)
`lab/alloy/config.alloy`	Pipeline variant exporting to local Tempo/Prometheus/Loki
`lab/grafana/provisioning/datasources/datasources.yml`	Trace↔logs↔metrics correlation settings to mirror in Grafana Cloud
`terraform/modules/span-metrics-alerts`	Health rules on span-metrics (PromQL from §4.2)
`terraform/modules/golden-signal-dashboard`	Dashboard using the same PromQL/TraceQL/LogQL
`agents/java.md` , `agents/dotnet.md` , `agents/python.md`	Full agent option lists per runtime